

EXPERIMENT: SUPERVISED CLASSIFICATION FOR DIABETES PREDICTION

Dataset: Pima Indians Diabetes Dataset

1. Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    roc_curve,
    classification_report
)
```

2. Load Dataset

```
# If using Google Colab, upload diabetes.csv first
# from google.colab import files
# uploaded = files.upload()

df = pd.read_csv("diabetes.csv")

print("First 5 rows of the dataset:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)
```

```
print("\nDataset Information:")
print(df.info())
```

3. Check Missing / Invalid Values

```
print("\nMissing Values:")
print(df.isnull().sum())

print("\nNumber of Zero Values:")
print((df == 0).sum())
```

4. Handle Invalid Zero Values

```
# Zero is medically unrealistic for these attributes:
# Glucose, BloodPressure, SkinThickness, Insulin, BMI

invalid_columns = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]

# Replace zero with NaN
df[invalid_columns] = df[invalid_columns].replace(0, np.nan)

print("\nMissing Values After Replacing Invalid Zeros:")
print(df.isnull().sum())
```

5. Separate Features and Target

```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

print("\nFeature Shape:", X.shape)
print("Target Shape:", y.shape)
```

6. Train-Test Split

```
# Stratified split maintains the same class proportion
# in training and testing datasets.

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y)
```

```
)

print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

print("\nTraining Class Distribution:")
print(y_train.value_counts())

print("\nTesting Class Distribution:")
print(y_test.value_counts())
```

7. Handle Missing Values

```
# Median imputation is suitable for numerical medical data.
# Fit only on training data to avoid data leakage.

imputer = SimpleImputer(strategy="median")

X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)
```

8. Feature Scaling

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train_imputed)
X_test_scaled = scaler.transform(X_test_imputed)
```

9. Create Classification Models

```
logistic_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

knn_model = KNeighborsClassifier(
    n_neighbors=5
)

decision_tree_model = DecisionTreeClassifier(
    random_state=42,
    max_depth=5
)
```

10. Train Logistic Regression

```
logistic_model.fit(X_train_scaled, y_train)
```

```
y_pred_lr = logistic_model.predict(X_test_scaled)
y_prob_lr = logistic_model.predict_proba(X_test_scaled)[: , 1]
```

11. Train K-Nearest Neighbors

```
knn_model.fit(X_train_scaled, y_train)

y_pred_knn = knn_model.predict(X_test_scaled)
y_prob_knn = knn_model.predict_proba(X_test_scaled)[: , 1]
```

12. Train Decision Tree

```
# Decision Tree does not require feature scaling,
# so use imputed but unscaled data.

decision_tree_model.fit(X_train_imputed, y_train)

y_pred_dt = decision_tree_model.predict(X_test_imputed)
y_prob_dt = decision_tree_model.predict_proba(X_test_imputed)[: , 1]
```

13. Evaluation Function

```
def evaluate_model(model_name, y_true, y_pred, y_prob):

    cm = confusion_matrix(y_true, y_pred)

    accuracy = accuracy_score(y_true, y_pred)

    precision = precision_score(
        y_true,
        y_pred,
        zero_division=0
    )

    recall = recall_score(
        y_true,
        y_pred,
        zero_division=0
    )

    f1 = f1_score(
        y_true,
        y_pred,
        zero_division=0
    )

    auc = roc_auc_score(y_true, y_prob)

    print("\n" + "=" * 60)
```

```

print(model_name)
print("=" * 60)

print("\nConfusion Matrix:")
print(cm)

print("\nAccuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))
print("ROC-AUC  :", round(auc, 4))

print("\nClassification Report:")
print(classification_report(
    y_true,
    y_pred,
    target_names=["Non-Diabetic", "Diabetic"],
    zero_division=0
))

return {
    "Model": model_name,
    "Accuracy": accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1-Score": f1,
    "ROC-AUC": auc
}

```

14. Evaluate All Models

```

results = []

results.append(
    evaluate_model(
        "Logistic Regression",
        y_test,
        y_pred_lr,
        y_prob_lr
    )
)

results.append(
    evaluate_model(
        "K-Nearest Neighbors",
        y_test,
        y_pred_knn,
        y_prob_knn
    )
)

```

```

results.append(
    evaluate_model(
        "Decision Tree",
        y_test,
        y_pred_dt,
        y_prob_dt
    )
)

```

15. Compare Model Performance

```

results_df = pd.DataFrame(results)

print("\n\nMODEL PERFORMANCE COMPARISON")
print("=" * 80)

print(
    results_df.round(4).to_string(index=False)
)

```

16. Visualize Performance Comparison

```

metrics = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1-Score",
    "ROC-AUC"
]

results_plot = results_df.set_index("Model")[metrics]

results_plot.plot(
    kind="bar",
    figsize=(12, 6)
)

plt.title("Comparison of Classification Models")
plt.ylabel("Score")
plt.xlabel("Classification Model")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend(loc="lower right")
plt.grid(axis="y", alpha=0.3)

plt.show()

```

17. Confusion Matrix - Logistic Regression

```
cm_lr = confusion_matrix(y_test, y_pred_lr)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_lr,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Non-Diabetic", "Diabetic"],
    yticklabels=["Non-Diabetic", "Diabetic"]
)

plt.title("Confusion Matrix - Logistic Regression")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```

18. Confusion Matrix - KNN

```
cm_knn = confusion_matrix(y_test, y_pred_knn)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_knn,
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=["Non-Diabetic", "Diabetic"],
    yticklabels=["Non-Diabetic", "Diabetic"]
)

plt.title("Confusion Matrix - KNN")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```

19. Confusion Matrix - Decision Tree

```
cm_dt = confusion_matrix(y_test, y_pred_dt)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_dt,
```

```

        annot=True,
        fmt="d",
        cmap="Oranges",
        xticklabels=["Non-Diabetic", "Diabetic"],
        yticklabels=["Non-Diabetic", "Diabetic"]
    )

plt.title("Confusion Matrix - Decision Tree")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()

```

20. ROC Curves

```

fpr_lr, tpr_lr, _ = roc_curve(y_test, y_prob_lr)
fpr_knn, tpr_knn, _ = roc_curve(y_test, y_prob_knn)
fpr_dt, tpr_dt, _ = roc_curve(y_test, y_prob_dt)

plt.figure(figsize=(8, 6))

plt.plot(
    fpr_lr,
    tpr_lr,
    label=f"Logistic Regression (AUC = {roc_auc_score(y_test, y_prob_lr):.3f})"
)

plt.plot(
    fpr_knn,
    tpr_knn,
    label=f"KNN (AUC = {roc_auc_score(y_test, y_prob_knn):.3f})"
)

plt.plot(
    fpr_dt,
    tpr_dt,
    label=f"Decision Tree (AUC = {roc_auc_score(y_test, y_prob_dt):.3f})"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

```



```
plt.title("ROC Curves for Classification Models")

plt.legend()
plt.grid(alpha=0.3)

plt.show()
```

21. Identify Best Model

```
best_model = results_df.loc[
    results_df["ROC-AUC"].idxmax()
]

print("\nBEST MODEL BASED ON ROC-AUC")
print("=" * 50)

print("Model      :", best_model["Model"])
print("Accuracy :", round(best_model["Accuracy"], 4))
print("Precision:", round(best_model["Precision"], 4))
print("Recall    :", round(best_model["Recall"], 4))
print("F1-Score  :", round(best_model["F1-Score"], 4))
print("ROC-AUC   :", round(best_model["ROC-AUC"], 4))
```